Republic of the Philippines
Bulacan State University
College of Industrial Technology
City of Malolos, Bulacan

IIDD 2 FINAL COURSE PROJECT

Individual Web Portfolio for Industry & OJT Placement
Project Documentation

Course Code & Title	CT 415, IIDD2
Project Name	FINAL DELIVERABLE IIDD2
Student Full Name	MARTIN, DUNCAN JAMES V.
Student ID	2023103642
Program & Section	BIT 4CG2
Instructor Name	Mrs. Vinna Ramos
Active Deployment URL	https://ojt-portfolio0.vercel.app/
Date of Submission	November 7, 2026

Section 1: Developer Profile & Links

Target Industry Track / Role	Junior Back-End Developer Intern
Live Deployment URL	https://ojt-portfolio0.vercel.app/
Public GitHub Repository Link	https://github.com/xortfnz/ojt-portfolio
Training Hours	600 hours required
Availability	Starting 2nd Semester, A.Y. 2027–2028

Professional Elevator Pitch

I am a Bachelor of Industrial Technology (Computer Technology) student at Bulacan State University with hands-on experience in web interface design, web system development, and ESP32 firmware programming. I work methodically, document what I build, and use AI tools to prototype quickly before testing and refactoring the code by hand. I want to bring dependable back-end logic and clear technical documentation to a partner company.

Cover Letter

November 7, 2026

The Hiring Manager / HR Lead
Partner Company

Subject: Application for On-the-Job Training – Junior Back-End Developer Intern

Dear Hiring Manager,

I am writing to apply for an On-the-Job Training (OJT) placement as a Junior Back-End Developer Intern. I am a fourth-year Bachelor of Industrial Technology student, major in Computer Technology, at the College of Industrial Technology, Bulacan State University, expecting to graduate in 2028. I will be available for deployment starting the 2nd Semester of A.Y. 2027–2028 and need to render 600 training hours.

During my studies I have designed and built practical systems. I redesigned the BulSU student portal as clickable Figma screens, and I developed the BulSU L&D Monitoring System, a web application that handles training requests through a four-step form, policy-rule validation, and a multi-level approval workflow. I have also programmed ESP32 firmware using the Arduino IDE, which taught me to work with hardware, networking, and logic that must behave reliably.

I use AI-assisted development tools to prototype quickly, but I always inspect, test, and refactor the output myself, and I record what I build and fix. My portfolio, available at https://ojt-portfolio0.vercel.app/, shows my projects and skills, and my source code is public at https://github.com/xortfnz/ojt-portfolio.

I would welcome the chance to discuss how I can contribute to your team. I can be reached at deejayzxcs@gmail.com. Thank you for your time and consideration.

Respectfully yours,

Duncan James V. Martin
BIT 4CG2, Bulacan State University – College of Industrial Technology

Section 2: Technology Stack & AI Tooling

1. Coding Environment & Core Stack

The portfolio was written in Visual Studio Code using plain HTML5 and CSS3 with no frameworks and no JavaScript dependencies. The page uses semantic elements (`header`, `nav`, `main`, `section`, `article`, `figure`, `footer`), CSS custom properties for the color palette, and Flexbox and CSS Grid for layout. Fonts (Archivo for headings and Public Sans for body text) are loaded from Google Fonts with system-font fallbacks. I used Google Chrome DevTools, especially device mode and the Elements and Console panels, to inspect the layout at different widths and to check for console errors. Git and GitHub were used for version control.

2. AI & Vibe Coding Tools Used

Claude (Anthropic) was my main AI development tool. I chose it because it follows detailed, multi-part instructions well, keeps a consistent structure across a long conversation, and can explain why a fix works, which helped me review the code instead of copying it blindly. I used it to scaffold the HTML structure for each required section, draft the stylesheet, and suggest refactors for mobile layouts. All generated code was read, tested in the browser, and edited by me before it went into the repository.

3. Deployment Platform

I deployed the site on Vercel, linked to the public GitHub repository. I chose Vercel because it publishes a live `.vercel.app` URL within a minute, redeploys automatically each time I push a commit to GitHub, and needs no build configuration for a static site. This made it easy to test fixes on the live URL right after changing the code.

Section 3: Prompt Engineering Strategies & Iteration Log

1. Prompting Strategies Applied

Technique 1: Context-setting. Before asking for any code, I gave the AI the complete requirements: the six required sections (Hero, About, Skills, Projects, Contact, Footer), my real details (name, target role, 600 training hours, availability window, degree, and projects), the color palette (navy text, a burnt-orange accent, and light blue-gray surfaces), the fonts, and the container width. Giving this context upfront meant the first draft already matched the content and look I wanted, and I did not have to correct invented details such as fake projects or wrong hours.

Technique 2: Constraint-based prompting. I set hard rules in the prompt, such as: "Use pure HTML5 and vanilla CSS only, no frameworks. Do not use inline styles. Use semantic tags. Use relative paths like assets/css/style.css, not absolute paths." These

constraints kept the code clean and made it compatible with the required folder structure and the deployment note on relative paths. When the AI broke a rule, I pointed to the exact rule it broke in my follow-up.

2. Prompt Iteration Log (Before and After)

Instance A (Layout / Component Issue)

Initial Prompt Given	*"Build a responsive one-page OJT portfolio for a BIT Computer Technology student with a hero, about, skills, projects, contact, and footer section."*
Flaw in AI Output	The first draft used fixed pixel widths for the two-column hero and the project cards, so on a phone-sized screen the cards were cut off and the page scrolled sideways. It also had several inline `style` attributes and only one breakpoint, so the tablet layout looked cramped.
Corrective Follow-Up Prompt / Strategy Used	*"Refactor this. Move every inline style into assets/css/style.css. Replace fixed widths with CSS Grid using repeat(auto-fit, minmax()). Add breakpoints at 900px and 560px: stack the hero and about columns at 900px, and make the metrics and project grids single-column at 560px. There must be no horizontal overflow at 360px width."* This constraint-based follow-up produced the stacked grids that are in the final stylesheet.

Instance B (Styling / Behavioral Issue)

Initial Prompt Given	*"Make the buttons and links look modern with hover effects."*
Flaw in AI Output	The AI added hover effects only. There were no pressed (active) states and no visible keyboard focus outline. The orange accent text was also too light against the pale background, which hurt readability, and the effects were not turned off for users who prefer reduced motion.
Corrective Follow-Up Prompt / Strategy Used	*"Use a darker accent (#a04a0c) so text meets WCAG AA contrast. Add :hover, :active, and :focus-visible states to all buttons and links. Keep transitions at about 0.15s. Add a prefers-reduced-motion rule that disables the transitions. Add a skip-to-content link."* The final CSS includes lift-on-hover and press-down-on-active button states, focus outlines, a skip link, and the reduced-motion rule.

Section 4: Real Bottlenecks & Practical Problem-Solving

Bottleneck 1: Layout & Mobile View Quirks

The part that broke on mobile screens was the hero metrics block and the contact details: long text such as the availability window and the email and LinkedIn addresses was wider than the screen and forced sideways scrolling. I diagnosed it in Chrome DevTools device mode by setting the width to 360px and using the Elements panel to find which element extended past the viewport. I fixed it by adding `overflow-wrap: anywhere` to the contact details and channel links, switching the metrics to a single column at 560px, and making the grids use `auto-fit` so they stack on their own.

Bottleneck 2: Live Deployment / Asset Loading Issues

Images are the part most likely to fail once the site is published. The required structure lists the project images as `project1.jpg` and `project2.jpg`, but my screenshots were saved as PNG files, so the file extension in `index.html` did not match the real file name. This kind of mismatch can go unnoticed on a local machine but causes broken images online, because Vercel serves files from a case-sensitive Linux server and will not find a file whose name or extension differs. I resolved it by comparing every `src` and `href` in the HTML against the actual files in the assets folder, making the extension in the HTML match the real file (or converting the screenshot to the expected format), and re-testing on the live URL with the DevTools Network and Console panels open until no 404 errors remained. I also kept every path relative (for example `assets/css/style.css`) instead of starting with a slash.

Bottleneck 3: Working with AI-Generated Code

The hardest part was keeping the code consistent. The AI would sometimes return a different class-naming style, add extra wrapper elements, or re-introduce a rule I had already removed when I asked for an unrelated change. It also wrote placeholder content when my details were not in the prompt, which I had to find and replace so that the portfolio stayed accurate. I handled this by reviewing each response line by line, keeping one naming convention, testing after every change instead of in large batches, and making small manual fixes in VS Code instead of asking the AI to regenerate whole files.

Section 5: Experience Reflection & OJT Readiness

1. Human vs. AI Roles

The AI did best at speed: it produced a semantic HTML skeleton, a full stylesheet with CSS variables, and grid layouts in minutes, and it explained what each rule did. It was weakest at details that depend on my real situation and on testing. It could not know my real projects, hours, or availability, it did not check that file names matched, and it

did not test the page on an actual phone. Manual intervention was needed to verify the content, debug the mobile overflow, fix asset paths, check accessibility, and make the final design decisions.

2. OJT Industry Readiness

The Featured Projects section best shows that I am ready for my training hours. It presents two real systems with different roles: the BulSU Website Redesign, where I worked as UI/UX Designer, and the BulSU L&D Monitoring System, where I worked as Developer. The second one covers form validation and approval workflows, which relate directly to the back-end internship role I am applying for. Together with the clear Contact section and the downloadable resume, a recruiter can see what I have built and how to reach me.

3. Future Upgrades

Before sending the portfolio to companies, I plan to add a source-code link and a short technical write-up to the L&D Monitoring System card, explaining the validation rules and approval flow, together with a back-end-focused mini project (such as a small database-driven API) so the portfolio matches my Junior Back-End Developer target more strongly.

Section 6: Proof of Deployment & Screenshots

The screenshots below show the live site at https://ojt-portfolio0.vercel.app/ and the public repository at https://github.com/xortfnz/ojt-portfolio.

Insert screenshot here
Figure 1. Full-page desktop view of the live website (URL bar visible)

Figure 1. Full-page desktop view of the live website (URL bar visible)

Insert screenshot here
Figure 2. Full-page mobile view of the live website (URL bar visible)

Figure 2. Full-page mobile view of the live website (URL bar visible)

Insert screenshot here
Public GitHub repository (URL bar visible)

Figure 3. Active public GitHub repository (URL bar visible)